

Tarefa 13 - Afinidade de Threads no Navier-Stokes

Objetivo

Avaliar como a escalabilidade do código de Navier-Stokes da Tarefa 12 muda quando alteramos a afinidade das threads no mesmo nó de computação do NPAD.

Como a Tarefa 12 mostrou que `omp-region` foi a versão mais consistente na escalabilidade forte, a Tarefa 13 usa essa versão como base e varia apenas a política de afinidade. Isso isola melhor o efeito de `OMP_PROC_BIND`, `OMP_PLACES` e `GOMP_CPU_AFFINITY`.

Configuração

- Modo do código: `omp-region`
- Malha fixa: `2048 x 2048`
- Passos de tempo: `1000`
- Escalonamento: `schedule(static)`
- Chunk: `0`
- Collapse: `1`
- Threads testadas: `1, 2, 4, 8, 16, 32`
- Afinidades testadas: `6`
- Rodadas coletadas: `108`
- Compilação: `gcc -O3 -march=native -fopenmp`

Políticas de afinidade

- `sem_bind`: `OMP_PROC_BIND=false`, sem `OMP_PLACES` explícito.
- `close_cores`: `OMP_PROC_BIND=close` e `OMP_PLACES=cores`.
- `spread_cores`: `OMP_PROC_BIND=spread` e `OMP_PLACES=cores`.
- `close_threads`: `OMP_PROC_BIND=close` e `OMP_PLACES=threads`.
- `spread_threads`: `OMP_PROC_BIND=spread` e `OMP_PLACES=threads`.
- `gomp_cpu_affinity`: usa `GOMP_CPU_AFFINITY` para listar explicitamente as CPUs disponíveis ao processo, uma extensão do runtime GNU OpenMP.

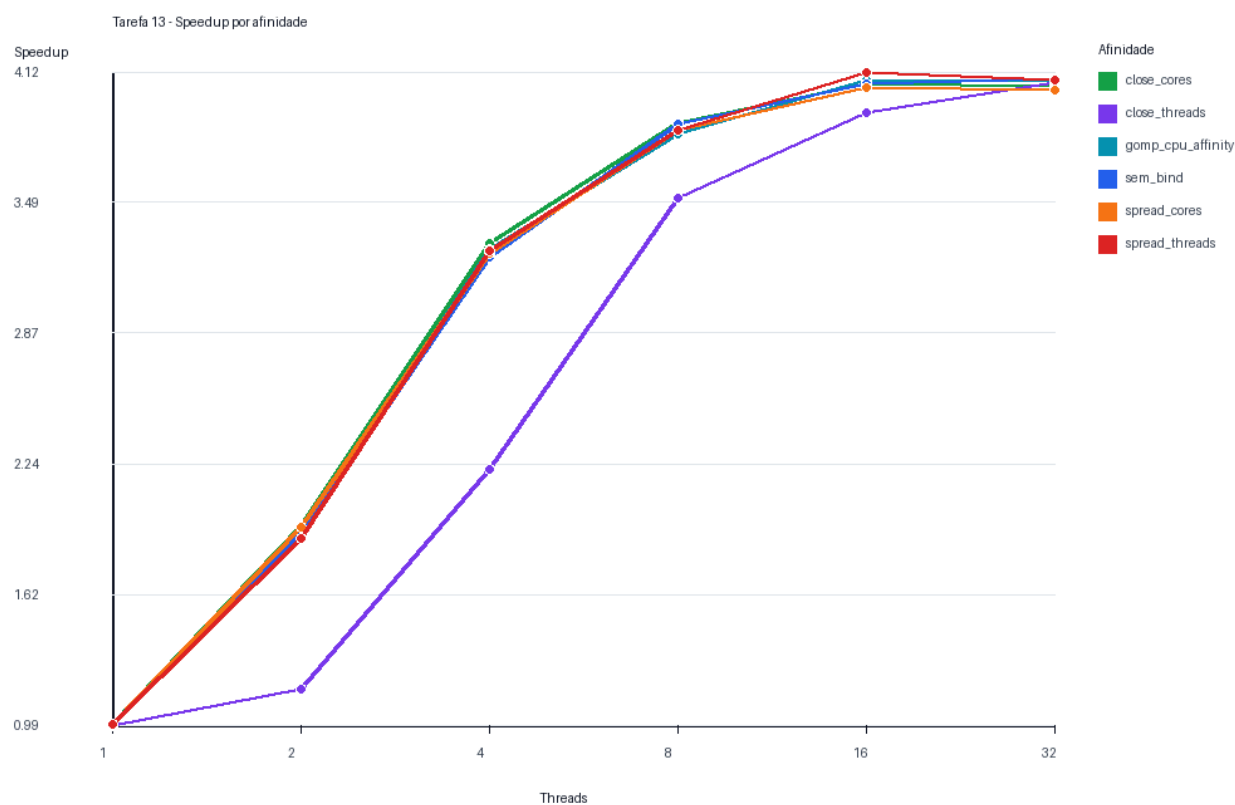
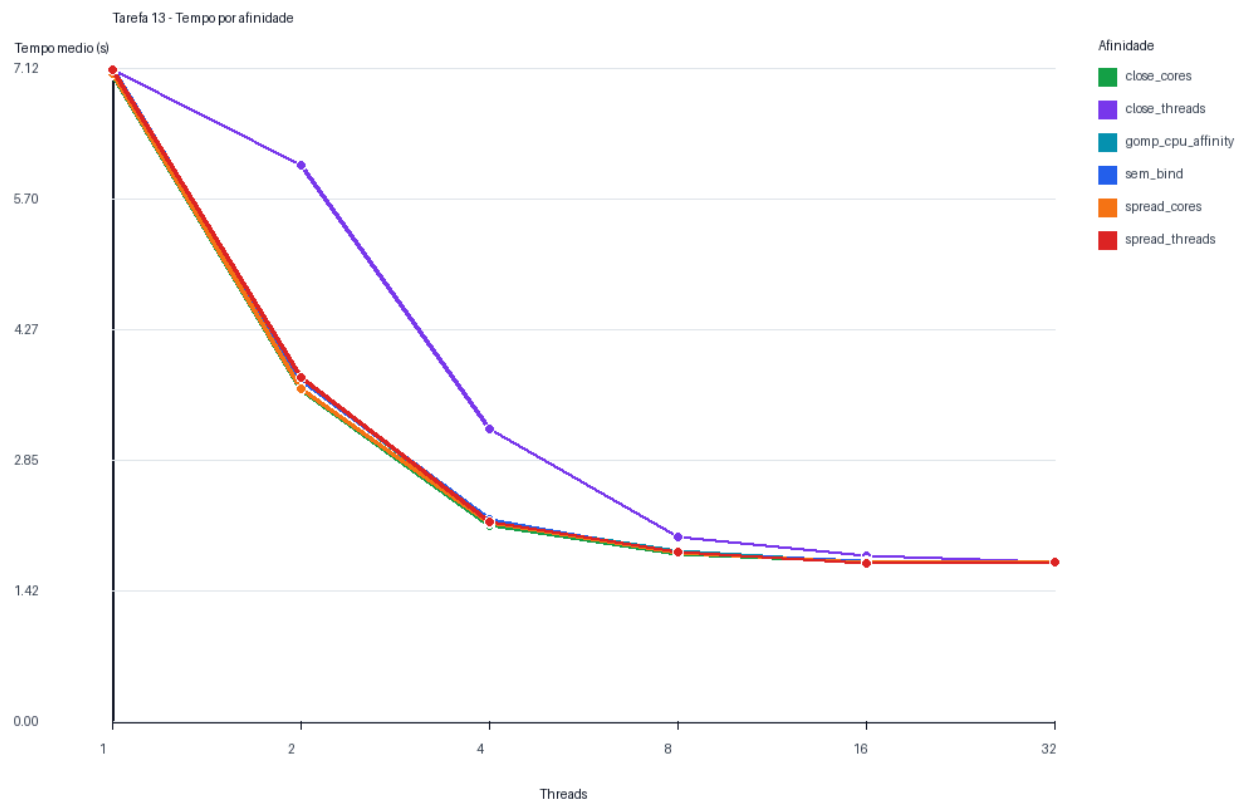
Validação numérica

O mesmo critério numérico da Tarefa 12 foi mantido: $dt * nu \leq 0.25$. A execução registrada iniciou com máximo `1.000000` e terminou com máximo `0.999256` no primeiro caso coletado. A norma L2 também foi registrada em todas as rodadas para confirmar que a mudança de afinidade não altera o resultado físico, apenas o tempo de execução.

Resultados

Afinidade	Threads	OMP_PROC_BIND	OMP_PLACES	Rodadas	Média (s)	Min (s)	Max (s)	Speedup	Eficiência
close_cores	1	close	cores	3	7.054435	7.041570	7.078738	1.00	1.00
close_cores	2	close	cores	3	3.615176	3.609295	3.619673	1.95	0.97
close_cores	4	close	cores	3	2.136567	2.133575	2.140000	3.30	0.82
close_cores	8	close	cores	3	1.817856	1.805036	1.838859	3.87	0.48
close_cores	16	close	cores	3	1.735049	1.730766	1.739202	4.06	0.25
close_cores	32	close	cores	3	1.740236	1.739996	1.740429	4.05	0.13
close_threads	1	close	threads	3	7.108468	7.061370	7.146916	0.99	0.99
close_threads	2	close	threads	3	6.061661	6.056218	6.064670	1.16	0.58
close_threads	4	close	threads	3	3.188963	3.187868	3.190675	2.21	0.55
close_threads	8	close	threads	3	2.010926	2.005471	2.019658	3.51	0.44
close_threads	16	close	threads	3	1.801664	1.797556	1.803991	3.92	0.24
close_threads	32	close	threads	3	1.735626	1.734961	1.736062	4.07	0.13

Afinidade	Threads	OMP_PROC_BIND	OMP_PLACES	Rodadas	Media (s)	Min (s)	Max (s)	Speedup	Eficiencia
gomp_cpu_affinity	1	-	-	3	7.102990	7.068519	7.165997	1.00	1.00
gomp_cpu_affinity	2	-	-	3	3.726741	3.722072	3.731056	1.90	0.95
gomp_cpu_affinity	4	-	-	3	2.164960	2.151888	2.178513	3.27	0.82
gomp_cpu_affinity	8	-	-	3	1.849523	1.848763	1.850192	3.82	0.48
gomp_cpu_affinity	16	-	-	3	1.731968	1.730508	1.733557	4.08	0.26
gomp_cpu_affinity	32	-	-	3	1.736284	1.736243	1.736358	4.07	0.13
sem_bind	1	false	-	3	7.123393	7.087278	7.177207	0.99	0.99
sem_bind	2	false	-	3	3.717072	3.686663	3.747390	1.91	0.95
sem_bind	4	false	-	3	2.192430	2.190027	2.194006	3.23	0.81
sem_bind	8	false	-	3	1.830975	1.828843	1.832696	3.87	0.48
sem_bind	16	false	-	3	1.744570	1.730988	1.754475	4.06	0.25
sem_bind	32	false	-	3	1.736150	1.735671	1.736468	4.08	0.13
spread_cores	1	spread	cores	3	7.056997	7.018930	7.084616	0.99	0.99
spread_cores	2	spread	cores	3	3.619500	3.573061	3.657120	1.94	0.97
spread_cores	4	spread	cores	3	2.161321	2.134826	2.210120	3.25	0.81
spread_cores	8	spread	cores	3	1.829668	1.803823	1.842835	3.84	0.48
spread_cores	16	spread	cores	3	1.736381	1.732204	1.740155	4.04	0.25
spread_cores	32	spread	cores	3	1.741618	1.741434	1.741759	4.03	0.13
spread_threads	1	spread	threads	3	7.108342	7.071734	7.139409	0.99	0.99
spread_threads	2	spread	threads	3	3.750598	3.740731	3.755898	1.89	0.94
spread_threads	4	spread	threads	3	2.169655	2.157935	2.175730	3.26	0.81
spread_threads	8	spread	threads	3	1.843063	1.821013	1.856067	3.84	0.48
spread_threads	16	spread	threads	3	1.717527	1.717233	1.717769	4.12	0.26
spread_threads	32	spread	threads	3	1.733905	1.728820	1.741087	4.08	0.13



Ranking por numero de threads

- 1 threads: melhor **close_cores** com media 7.054435s; pior **sem_bind** com media 7.123393s; diferenca de 1.0%.
- 2 threads: melhor **close_cores** com media 3.615176s; pior **close_threads** com media 6.061661s; diferenca de 67.7%.
- 4 threads: melhor **close_cores** com media 2.136567s; pior **close_threads** com media 3.188963s; diferenca de 49.3%.
- 8 threads: melhor **close_cores** com media 1.817856s; pior **close_threads** com media 2.010926s; diferenca de 10.6%.
- 16 threads: melhor **spread_threads** com media 1.717527s; pior **close_threads** com media 1.801664s; diferenca de 4.9%.
- 32 threads: melhor **spread_threads** com media 1.733905s; pior **spread_cores** com media 1.741618s; diferenca de 0.4%.

Analise

O melhor caso agregado foi `spread_threads` com 16 threads, media de 1.717527s e speedup de 4.12x dentro da propria politica de afinidade.

As politicas `close` tendem a favorecer localidade de cache, porque mantem threads em posicoes proximas. Isso pode ajudar quando o trabalho compartilha dados proximos na memoria. As politicas `spread` tendem a distribuir threads pelo no, o que pode reduzir competicao local por recursos de um mesmo nucleo fisico ou socket. Para este stencil 2D, que faz poucos calculos por celula e muitos acessos a memoria, o resultado tende a depender fortemente da largura de banda de memoria e da topologia do no.

Na Tarefa 12, o desempenho saturou depois de 8 a 16 threads. A Tarefa 13 verifica se essa saturacao muda quando o runtime fixa as threads em nucleos proximos, espalha as threads pelo no ou deixa o sistema operacional migrar threads. Se `sem_bind` for pior, isso indica custo de migracao e perda de localidade. Se `spread_cores` for melhor em altas contagens de threads, isso sugere que distribuir o acesso a memoria e aos caches do no foi mais importante que manter as threads proximas.

Artefatos

- Codigo: `Tarefa-13/navier_scaling.c`
- Coleta: `Tarefa-13/coletar_afinidade.py`
- CSV: `Tarefa-13/resultados/tarefa13_afinidade.csv`
- Graficos: `Tarefa-13/resultados/affinity_elapsed.png` e `Tarefa-13/resultados/affinity_speedup.png`
- Relatorio: `Tarefa-13/resultados/relatorio_tarefa13.md`